Logical View

The logical view captures the system's static structure regarding classes or domain concepts and their collaborations. It supports the specification of services, behaviors, and responsibilities related to functional requirements. UML class diagrams or ER diagrams are commonly used to express this view.

- **Purpose**: To describe the system's functional structure using object-oriented or domain-driven abstractions.
- **Key Stakeholders**: Software designers, application developers.
- **Concerns**:
    - Classes, entities, and their relationships (e.g., inheritance, associations).
    - Domain modeling and functional decomposition.
    - Reuse of design patterns and encapsulation of logic.
- **What is irrelevant**:
    - Execution environment or process scheduling.
    - Source code file structure or physical deployment.

Process View

The process view addresses non-functional properties such as scalability and responsiveness. It identifies which components are active concurrently and how they interact at runtime. This view is essential for ensuring that timing, throughput, and fault-isolation requirements are met.

- **Purpose**: To model the dynamic aspects of the system, including process concurrency and inter-process communication.
- **Key Stakeholders**: Systems engineers, performance analysts, architects.
- **Concerns**:
    - Processes, threads, and their synchronization.
    - Inter-process communication mechanisms (e.g., queues, events, RPC).
    - Runtime performance, fault tolerance, and load distribution.
- **What is irrelevant**:
    - Source code organization or packaging.
    - Deployment hardware configurations.

Development View

The development view focuses on how the system is decomposed for implementation. It represents the structure of the source code in terms of packages or components and their

interdependencies. This view supports team collaboration, configuration management, and system evolution. Tools like IDEs or build systems often reflect this view directly.

- **Purpose**: To describe the software's modular structure and organization for efficient development.
- **Key Stakeholders**: Developers, build engineers.
- **Concerns**:
    - Modules, packages, or components and their dependencies.
    - Layered architecture or build-time structure.
    - Source control and integration guidelines.
- **What is irrelevant**:
    - Runtime behavior or deployment information.
    - Detailed user interactions or workflows.

Physical View

The physical view outlines how the software system is distributed across computing resources. It describes deployment configurations, including node assignments, hardware constraints, and communication links. This view is essential for planning installation, load balancing, failover, and scalability in production environments.

- **Purpose**: To represent the deployment of software components onto physical hardware infrastructure.
- **Key Stakeholders**: System administrators, network architects, operations engineers.
- **Concerns**:
    - Mapping of software artifacts to servers or nodes.
    - Communication paths, network topology, and redundancy.
    - Performance and availability at the infrastructure level.
- **What is irrelevant**:
    - Source code organization or class structure.
    - Logical design or abstract functional decomposition.

Use Case View ("+1")

The scenarios view connects the four main views by demonstrating how system components collaborate to fulfill key requirements. It supports validation and communication by using specific cases to verify the completeness and soundness of the architecture. Tools such as sequence diagrams or scenario scripts are typically used here.

- **Purpose**: To illustrate the interaction of architectural elements across views using representative use cases.
- **Key Stakeholders**: All stakeholders, including users, testers, architects, and reviewers.
- **Concerns**:
    - End-to-end workflows or usage scenarios.
    - Interaction sequences that validate the architecture.
    - Architectural prototypes and regression scenarios.
- **What is irrelevant**:
    - Complete coverage of all functions.
    - Deep internals of individual components.

SOURCE